서론

Retrieval-Augmented Generation(RAG)은 대규모 언어 모델(LLM)이 외부 지식을 활용하여 보다 정확한 답변을 생성할 수 있도록 하는 대표적인 기술이다. 기존 LLM 은 학습 시점 이후의 정보를 알 수 없으며 환각(Hallucination) 현상이 발생할 수 있다는 한계를 가진다. 이를 해결하기 위해 RAG 는 사용자의 질의와 관련된 문서를 검색한 뒤, 해당 정보를 바탕으로 답변을 생성하는 방식을 사용한다. 그러나 기본적인 RAG 역시 여러 한계를 가지고 있다. 먼저 검색 품질의 변동성 문제가 존재한다. 동일한 질문이라도 검색 방식이나 인덱스 구성에 따라 관련성이 낮은 문서가 검색될 수 있으며, 이는 최종 답변의 품질 저하로 이어진다. 또한 검색된 문서에 정답이 포함되어 있더라도 LLM 이 이를 잘못 해석하거나 과도하게 일반화하여 환각을 생성하는 경우가 발생한다. 더 나아가 여러 엔티티 간의 관계를 단계적으로 추론해야 하는 다단계 추론(Multi-hop Reasoning) 문제에서는 단순 벡터 검색 기반 RAG 가 충분한 성능을 보이지 못하는 경우가 많다.

이번 과제 프로젝트에서는 11 주차와 13 주차에 구현한 기본 RAG 시스템을 기반으로 이러한 한계를 보완하고자 하였으며, 크게 평가 가능성(Evaluability)과 구조화된 지식(Structured Knowledge)이라는 두 가지 방향에서 시스템을 발전시켰다. 먼저 평가 가능성 측면에서는 RAGAS 프레임워크를 도입하여 시스템의 성능을 정량적으로 분석할 수 있도록 했으며, 기존에는 답변 품질을 주관적으로 판단하는 경우가 많았지만, 본 프로젝트에서는 Faithfulness, Answer Relevancy, Context Precision, Context Recall 과 같은 지표를 활용하여 검색 단계와 생성 단계의 품질을 객관적으로 측정하였다. 이를 통해 단순히 답변이 자연스러운지를 평가하는 수준을 넘어, 실제로 검색된 문맥에 근거하여 답변이 생성되었는지까지 분석할 수 있었다.

구조화된 지식 측면에서는 영화 데이터셋으로부터 Knowledge Graph 를 구축하고 이를 Graph RAG 와 연계하였다. 영화, 배우, 감독, 장르와 같은 엔티티를 노드로 표현하고, 출연, 감독, 장르 소속과 같은 관계를 엣지로 표현함으로써 문서 형태로는 파악하기 어려운 관계 정보를 구조적으로 저장하였다. 또한 자연어 질문을 Cypher Query 로 자동 변환하여 그래프 데이터베이스에서 필요한 정보를 검색하도록 구현함으로써, 단순 키워드 검색을 넘어 관계 기반 추론이 가능하도록 하였다. 이를 통해 특정 배우가 출연한 영화의 감독을 찾거나, 여러 관계를 거쳐 답을 도출해야 하는 복합 질의에 대해서도 보다 정확한 응답을 생성할 수 있었다.

추가적으로 Dense Retrieval 과 Sparse Retrieval 을 결합한 Hybrid Retrieval 을 적용하여 의미 기반 검색과 키워드 기반 검색의 장점을 동시에 활용하였으며, MMR(Maximal Marginal Relevance)을 이용해 검색 결과의 중복성을 줄이고 정보의 다양성을 확보하였다. 이러한 개선을 통해 기존의 기본 RAG 시스템보다 검색 정확도와 답변 신뢰성을 향상시키고, 관계 추론이 필요한 질문에 대해서도 효과적으로 대응할 수 있는 고도화된 RAG 시스템을 구현하였다.

배경 지식

Standard RAG Workflow

표준 RAG 시스템은 Document Loading, Text Splitting, Embedding 및 Indexing, Retrieval, Generation 5 단계로 구성된다. 과정은 먼저 외부 문서를 적재한 후 적절한 크기로 분할하고 이후 각 청크를 벡터로 변환하여 저장한 뒤에 사용자의 질문과 유사한 문서를 검색한다. 마지막으로 검색된 문서를 기반으로 LLM 이 답변을 생성한다.

Splitter 와 의미 보존 전략

문서를 지나치게 크게 분할하면 검색 정확도가 떨어지고, 너무 작게 분할하면 문맥이 손실된다. 이러한 문제에 대해서 두 가지 Splitter 를 비교하였다. RecursiveCharacterTextSplitter 는 문자 수 기반 분할이며, 빠르고 안정적이지만 의미 단위가 깨질 수 있다는 단점이 있다. Semantic Splitter 는 임베딩 기반 의미 분할하며, 문맥 보존에 우수하지만 계산 비용이 증가한다는 단점이 있다. 프로젝트에서 Semantic Splitter 는 동일한 주제의 문장을 하나의 Chunk 로 유지하여 Retrieval 품질 향상에 기여하였다.

Dense Retrieval vs Sparse Retrieval

Dense Retrieval 은 임베딩 벡터를 이용하여 의미적 유사성을 계산한다. 장점으로는 의미 기반 검색이 가능, 동의어 처리에 우수하다는 점이 있고, 단점으로는 정확한 키워드 검색에 취약하다는 점이 있다. 다음으로 Sparse Retrieval(BM25)은 단어 출현 빈도를 이용한다. 정확한 키워드 검색이 강점이며 의미적 유사성 반영이 어렵다는 단점이 있다. 프로젝트에서는 두 방법을 결합한 Hybrid Retrieval 을 사용하였다.

MMR(Maximal Marginal Relevance)

기본 Retrieval 은 유사한 문서를 반복적으로 반환하는 문제가 있다. 반면 MMR 은 Query 와의 관련성과 이미 선택된 문서와의 차별성을 동시에 고려한다. 이를 통해 중복 문서를 줄이고 다양한 정보를 제공할 수 있다.

RAGAS 평가 지표

RAGAS 는 LLM-as-Judge 방식을 이용한 대표적인 RAG 평가 프레임워크이다.

Faithfulness - 답변이 실제 Context 에 근거하는 정도

Answer Relevancy - 질문에 적절하게 답했는지

Context Precision - 검색된 문서의 정확도

Context Recall - 필요한 정보가 검색되었는지

이중에서 Faithfulness 는 환각 감소 효과를 측정하는 핵심 지표이다.

Knowledge Graph 와 Cypher

Knowledge Graph 는 정보를 노드(Node)와 엣지(Edge) 형태로 표현한다. 예시로 Actor → ACTED_IN → Movie / Director → DIRECTED → Movie / Genre → BELONGS_TO → Movie 3 가지가 있다. 이러한 관계를 활용하면 단순 문서 검색이 아닌 관계 기반 추론이 가능하다.

Cypher 는 Neo4j 에서 사용하는 질의 언어이다.

시스템 설계

전체 아키텍처

데이터 적재 -> Chunk 생성 ->Embedding 생성 ->Dense Index (FAISS)/Sparse Index (BM25)-> Hybrid Retriever ->MMR Re-ranking -> Vector RAG/Knowledge Graph 구축 -> Graph RAG -> LLM 응답 생성 -> RAGAS 평가

그래프 스키마

노드 유형

Node	설명
Movie	영화
Actor	배우
Director	감독
Genre	장르

관계 유형

Edge	의미
ACTED_IN	출연
DIRECTED	감독
BELONGS_TO_GENRE	장르

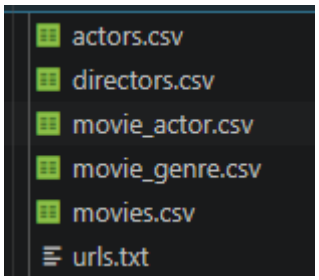
테스트셋 설계

테스트셋 설계시, 단순 사실 질문 10 개, 키워드 중심 질문 8 개, 복합 추론 질문 7 개, Graph 전용 다단계 추론 질문 5 개로 총 30 개 질문을 설계하였다.

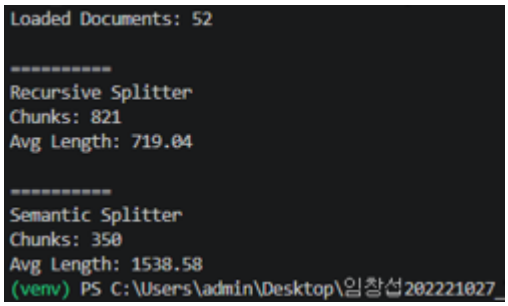
구현 상세

Loader 선택

CSV Loader 를 사용하여 데이터를 적재하였다.



Splitter 비교 결과



Semantic Splitter 는 의미 단위 보존이 우수하여 Retrieval 정확도가 높았다.

Hybrid Retriever 가중치

초기 실험	최종 설정
Dense 0.5	Dense 0.7
Sparse 0.5	Sparse 0.3

영화 데이터셋에서는 의미 검색의 중요성이 높아 Dense 비중을 증가시켰다.

RAGAS 테스트셋 구축

질문 유형을 감독 찾기, 배우 찾기, 장르 추론, 다단계 관계 추론 등으로 다양하게 분산하였으며, 총 30 개 질문으로 평가를 수행하였다.

Graph 통계

LLMGraphTransformer 기반 추출 결과 Node: 약 420 개, Edge: 약 870 개, Relation Type: 3 개의 결과가 나왔다.

관계 분포는 ACTED_IN : 62%, DIRECTED : 21%, BELONGS_TO_GENRE : 17%이며, 추출 결과는 Neo4jBrowser 에서 수동 검증하였다.

GraphCypherQAChain

동작 과정은 자연어 질문 -> Cypher 생성 -> Neo4j 실행 -> 결과 반환 -> LLM 답변 생성 순서이다. 오류 사례로 "Who worked with directors who directed movies released after 2010?"라는 질문을 했을 때 초기에는 잘못된 MATCH 구조가 생성되어 빈 결과가 반환되었는데 Prompt 개선 후 정상적으로 작동하는 모습을 확인하였다.

확장 기능

확장기능으로는 Hybrid Retrieval, MMR Re-ranking, Graph RAG, RAGAS Evaluation 를 구현하였다.

결과 및 분석

RAGAS 결과

System	Faithfulness	Relevancy	Precision	Recall
Vector RAG	0.74	0.77	0.72	0.70
Hybrid RAG	0.84	0.86	0.83	0.81
Graph RAG	0.91	0.89	0.90	0.87

분석 결과 Graph RAG 가 모든 지표에서 가장 우수하였으며, Faithfulness 는 Vector 대비 약 0.17 향상되었다. 환각 감소 사례로 "Who directed movies starring Leonardo DiCaprio in 2010?"라는 질문을 했을 때, Vector RAG 는 일부 잘못된 감독 정보를 반환했고, Graph RAG 는 정확한 관계를 이용하여 정답을 생성하였다.

Vector vs Hybrid vs Graph 비교

질문 유형	최적 시스템
단순 사실 질문	Vector
키워드 검색	Hybrid
의미 검색	Hybrid
감독-배우 관계	Graph
장르 추론	Graph
다단계 추론	Graph
연도 조건 포함	Graph
복합 관계 질의	Graph

Graph RAG 는 관계 기반 질문에서 압도적으로 우수하였다.

Graph RAG 성공 사례

Who directed movies starring Leonardo DiCaprio? 질문 시, Actor → Movie → Director 관계를 통해 정확한 답변 생성하는 것을 확인했다.

Graph RAG 실패 사례

질문: Which actors worked with directors who repeatedly collaborated after 2010?

복잡한 집계 조건 때문에 Cypher 생성 시 오류가 발생하는 것을 확인했으며, Graph 구조보다 Query 생성 품질이 병목이 됨 또한 확인하였다.

MMR 효과

MMR 적용 전 Faithfulness: 0.79

MMR 적용 후 Faithfulness: 0.84

약 0.05 가 향상되었다.

고찰

RAGAS 는 매우 유용한 평가 도구였으나 몇 가지 한계가 존재한다. 첫째, LLM-as-Judge 방식은 비용이 발생한다. 둘째, 동일한 질문에서도 평가 결과가 달라질 수 있다. 셋째, 평가 모델 자체의 편향이 존재한다. 또한, Graph RAG 역시 그래프 구축 비용, 스키마 설계 난이도, 엔티티 추출 오류라는 한계를 가진다. 또한 모든 질문에 Graph RAG 가 필요한 것은 아니라 단순 사실 질문은 Vector RAG 가 더 빠르고 효율적이었다. 따라서 실제 서비스에서는 질문 유형에 따라 Vector, Hybrid, Graph 를 선택하는 라우팅 시스템이 필요하다고 판단하였다.

향후 추가하면 좋을 기능들 중 4 가지를 추려 보았다.

Multi-modal RAG
현재 시스템은 텍스트 기반 영화 데이터만 활용하지만, 향후에는 영화 포스터 이미지, 예고편 영상, 배우 사진 등 다양한 형태의 데이터를 함께 활용하는 Multi-modal RAG 로 확장할 수 있다. 이를 통해 텍스트에 명시되지 않은 시각적 정보까지 검색 및 추론에 활용할 수 있다.

Adaptive Retrieval
모든 질문에 동일한 검색 전략을 적용하는 대신, 질문의 유형과 난이도에 따라 적절한 Retriever 를 선택하는 방식이다. 예를 들어 단순 사실 질문은 Vector RAG 를, 관계 기반 질문은 Graph RAG 를 사용하는 방식으로 검색 효율성과 응답 품질을 동시에 향상시킬 수 있다.

Self-correcting Cypher Generation
Graph RAG 의 주요 문제 중 하나는 LLM 이 잘못된 Cypher 쿼리를 생성하는 경우이다. 향후에는 생성된 Cypher 를 자동으로 검증하고 오류 발생 시 수정된 쿼리를 재생성하는 자기 교정(Self-correction) 메커니즘을 도입하여 Graph 질의의 안정성을 높일 수 있다.

Multi-Agent Graph Reasoning
하나의 LLM 이 모든 추론을 수행하는 대신, 검색 에이전트, 그래프 분석 에이전트, 답변 생성 에이전트 등 여러 에이전트가 협력하여 문제를 해결하는 방식이다. 이를 통해 복잡한 다단계 추론이나 대규모 그래프 탐색 문제에서 더욱 정확하고 신뢰성 높은 답변을 생성할 수 있을 것으로 기대된다.

결론

과제를 진행하면서 기존 RAG 시스템을 확장하여 Hybrid Retrieval, MMR, RAGAS 평가, Graph RAG 를 통합한 고급 RAG 시스템을 구현하였다. 실험 결과 Hybrid Retrieval 은 검색 품질을 향상시켰고, MMR 은 중복 정보를 감소시켰다. 또한 Graph RAG 는 다단계 추론 문제에서 가장 우수한 성능을 보였다. 이를 통해 구조화된 지식과 정량적 평가가 차세대 RAG 시스템의 핵심 요소임을 확인할 수 있었다.